# Closing the XML Gap in TypeScript (for Localization Workflows)

Rodolfo M. Raya **Maxprograms** rmraya@maxprograms.com
17 June 2026
© 2026 Maxprograms

*Describes the move from Java to TypeScript for XML processing in localization workflows, and the TypesXML, TypesBCP47, and TypesXLIFF libraries built to bring DTD and XML Schema validation for TMX, XLIFF, and SRX to Node.js.*

## Introduction

If you've ever tried to process XML in Node.js or TypeScript, you've probably run into a limitation: parsing is easy, but validation against DTD or XML Schema is not.

That becomes a real problem when working with localization standards like TMX, XLIFF or SRX, where structure and validity are not optional. They define whether your data is usable.

This is the gap I ran into, and eventually decided to close.

## The old setup: Java for XML

Localization standards such as TMX, XLIFF, SRX and TBX are all XML-based. When I started working with them more than 20 years ago, Java was the most practical choice. Its XML tooling wasn't perfect, but it was reliable enough for parsing and validation.

That worked well for a long time.

The problem wasn't XML; it was everything around it.

Java desktop UI options didn't evolve in a way that fit my needs. I used SWT for years, but it became less maintained, and alternatives like Swing or JavaFX didn't appeal to me.

## Moving to TypeScript

Around 2018, I shifted toward building user interfaces with HTML, CSS and JavaScript. Eventually, TypeScript became the obvious choice because it provides a safer development model with static checks.

That solved the UI side.

But it created a new problem:

- UI in TypeScript
- XML processing still in Java

Maintaining two stacks for a single workflow wasn't ideal.

## The missing piece: XML validation in TypeScript

At the time, there were XML parsers in JavaScript, but none supported:

- DTD validation
- XML Schema validation

For localization formats, that's a hard requirement. Without validation, you can't guarantee that a TMX or XLIFF file is structurally correct.

So I started building **TypesXML**, a native XML parser and validator for TypeScript.

## A minimal example

Here's what basic XML parsing looks like:

```
import { Catalog, DOMBuilder, SAXParser } from "typesxml";

const parser: SAXParser = new SAXParser();
const handler: DOMBuilder = new DOMBuilder();
const catalog = new Catalog('/pathTo/catalog.xml');

parser.setCatalog(catalog);
parser.setContentHandler(handler);
parser.setValidating(true);
parser.parseFile("example.xml");

const document = handler.getDocument();
console.log(document.toString());
```

The same parser can be configured to:

- validate against DTD
- validate against XML Schema
- resolve XML catalogs

This makes it possible to process XML in Node.js with the same level of control traditionally associated with Java libraries.

## Building the missing stack

Creating a full XML stack took time:

- DOM and SAX-style parsing came first
- DTD validation followed
- XML Schema validation was the final step

XML Schema support is significantly more complex, but it's essential for formats like XLIFF.

## Supporting pieces: language data

Localization workflows also depend heavily on language codes.

BCP47 defines the standard, but the official data is distributed as a large text file. To make it easier to use in TypeScript, I built **TypesBCP47**, a companion library that parses that data and exposes it in a structured way.

## Proof of concept: rewriting existing tools

To validate the approach, I rewrote existing tools in TypeScript:

- **SRXEditor**
- **TMXValidator**

These tools:

- process large XML files
- validate against official grammars
- maintain performance comparable to the original Java versions

That confirmed that TypeScript can handle real-world localization workloads.

## XLIFF support

XLIFF uses XML Schema, which required implementing a full schema validator.

With that in place, I built **TypesXLIFF**, a library for parsing, generating, and validating XLIFF 2.x files.

It supports:

- XML Schema validation
- validation against the XLIFF specification
- a typed object model
- JSON conversion

## What this enables

With these pieces in place, it's now possible to:

- Validate TMX, XLIFF and SRX directly in Node.js
- Build localization pipelines entirely in TypeScript
- Remove the need for a separate Java backend
- Keep both UI and processing in the same language

## What's next

TBX support is in progress, along with updates to other tools to move them fully to TypeScript.

Some Java code still exists in older components, but new development is now centered on TypeScript.

## Closing thoughts

For a long time, XML processing in localization workflows effectively required Java.

That's no longer the case.

If you're working with XML-based formats in Node.js, especially in localization or structured data pipelines, you can now handle parsing, validation, and processing entirely in TypeScript.

## About the author

Rodolfo M. Raya, CTO of Maxprograms SAS, is involved in the development of localization standards including XLIFF, TMX, TBX, SRX and GlossML.

He develops tools and libraries that implement these specifications, including the software and validation services available on this site.